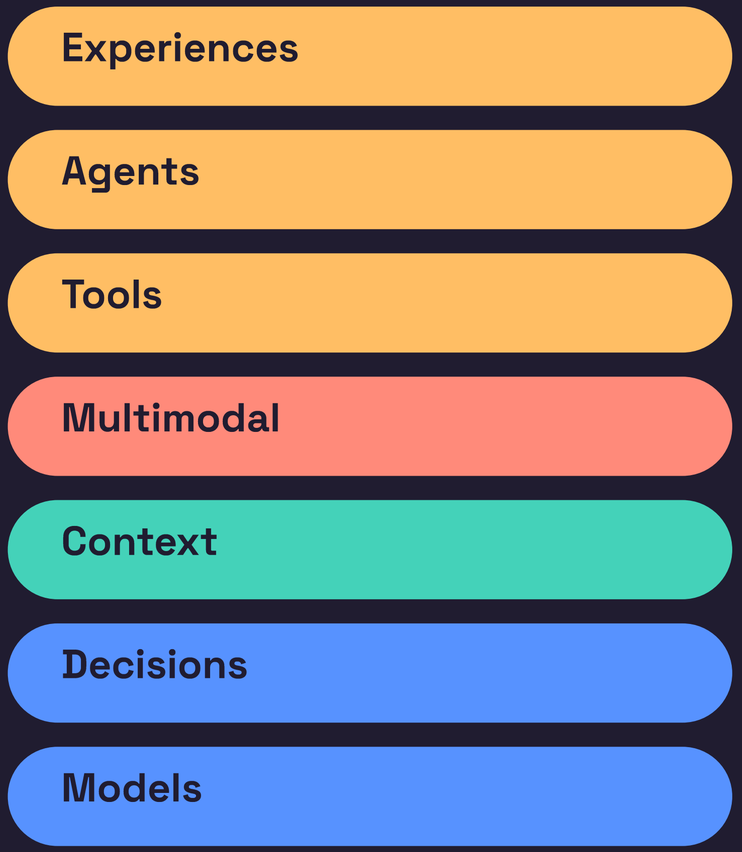
.NET CONF 2026

The .NET AI Stack

What's New, What's Next, and How It Fits Together

Luis Quintanilla



Building blocks. A growing ecosystem.

Build across the stack. Connect through shared foundations.

170M+ package downloads | Hundreds of dependent packages

User experiences

AG-UI for .NET

Blazor AI Components

Collaborative interfaces
Blazor components: experimental

Agents

Microsoft Agent Framework

GitHub Copilot SDK for .NET

Orchestration + agent-powered apps

Models

OpenAI .NET SDK

Anthropic C# SDK

OllamaSharp

Cloud + local models through .NET integrations

Data

Community Toolkit for .NET AI

DocumentExtraction Client

Vector connectors and the broader toolkit

Tools

MCP C# SDK

Connect application tools to agents + other apps

Evaluation

Microsoft.Extensions.AI.Evaluation

AgentEval

Response quality + agent behavior
AgentEval: community

SHARED FOUNDATIONS

Microsoft.Extensions.AI

Model interaction + embeddings
Tool definitions + invocation

Microsoft.Extensions.VectorData

Vector storage + search abstractions
Optional MEAI embedding generation

Microsoft.Extensions.DataIngestion

Document ingestion pipelines (preview)
Compose readers, processors + writers

Accents show selected shared-library integrations, including adapters; not exhaustive dependencies. Metrics: Nov 3, 2026.

What’s new in .NET + AI

New capabilities. More choice. Familiar .NET skills.

Official OpenAI / Anthropic SDK MEAI integrations

Use familiar MEAI contracts with the official SDKs. Reuse application code while retaining provider choice.

TRY STARTER KIT | aka.ms/meai/start

DocumentExtractionClient

Bring documents into your workflow. Use a common extraction contract with a choice of providers.

PREVIEW / EXPERIMENTAL

RoutingChatClient

Different tasks need different intelligence. Choose chat models using your own application policies.

EXPERIMENTAL API

OpenAI Decisions API

Sometimes you need a decision, not a conversation. Get probabilities, choices and scores.

EXPERIMENTAL
EXTENSIONS AI DECISION API (coming soon)

AG-UI + Blazor AI Components

Move beyond a chat box.
Build interactive experiences where users and agents work together.

BLAZOR AI COMPONENTS: EXPERIMENTAL

MCP C# SDK 2.0

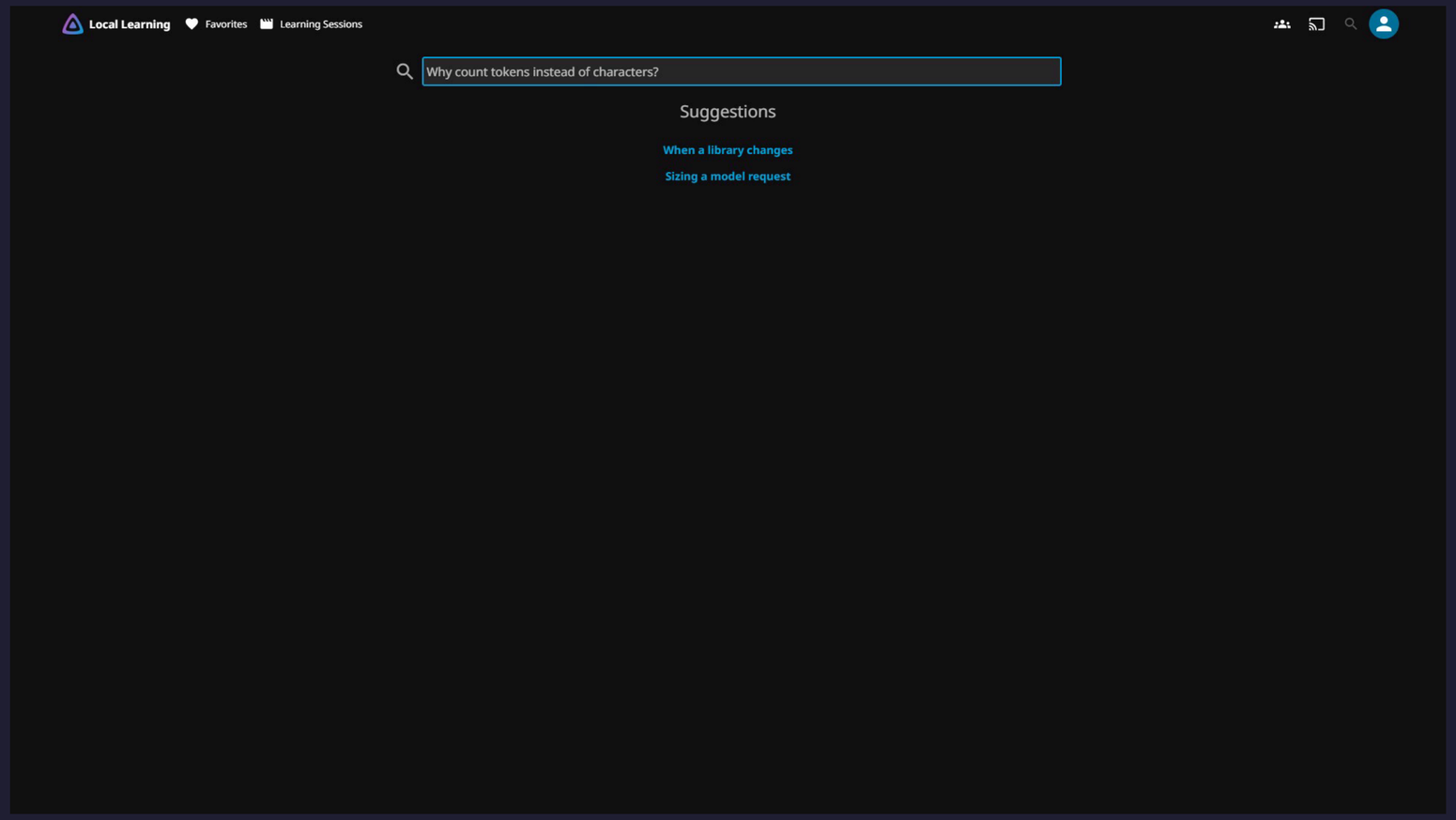
Connect application tools to agents and other apps. Put existing capabilities to work in new workflows.

MCP C# SDK | 2.0 MAJOR MILESTONE

Experimental APIs may change.

Your app. New possibilities.

Add intelligence without starting over.



Keep the application

Jellyfin stays Jellyfin

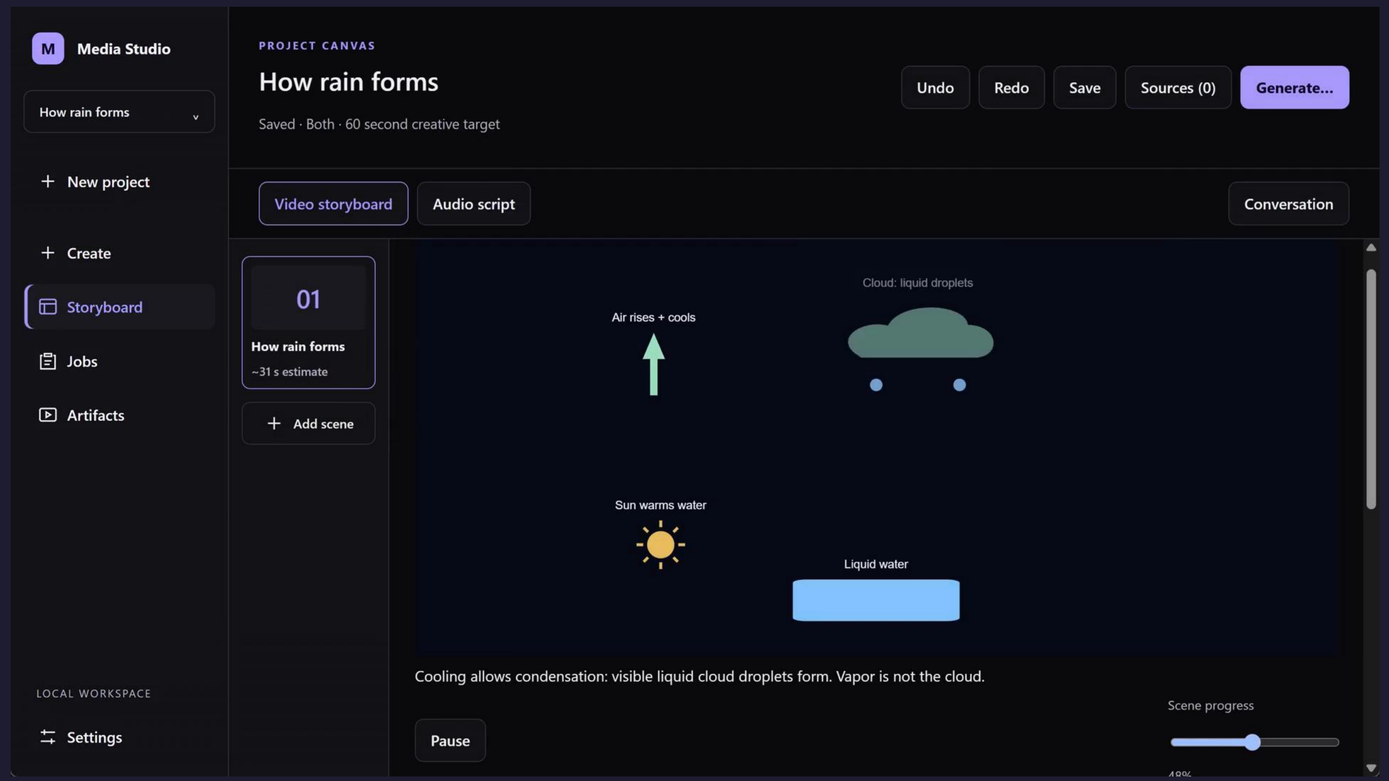
Add a new capability

Search by meaning

Existing application + a new capability. Click the authentic search/playback clip.

Beyond chat. From intent to action.

Compose capabilities. Collaborate with users.



Storyboarding | Narration | Captions + transcription
Animations | Thumbnails | Rendering

Microsoft.Extensions.AI + app-owned services. Code for deterministic steps.
Blazor AI Components + RoutingChatClient are experimental; capability/API availability varies.

Rich, collaborative UX

AG-UI + Blazor AI Components
Review, edit and approve

Choose the right chat model

Local/cloud chat models
RoutingChatClient + your policies

Let’s build one.

A research paper in. A narrated explainer out. One .NET app.

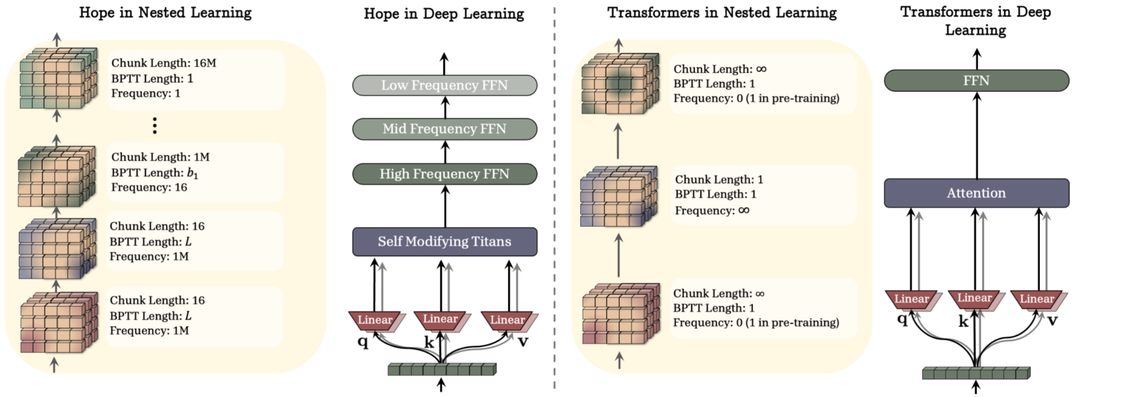


Figure 5: A comparison of Hope architectural backbone with Transformers (Normalization and potential data-dependent components are removed for the sake of clarity).

1 **Bring the paper**
PDF + figures + tables

2 **Read it**
Sections, tables, Figure 5

3 **Draft with an agent**
Typed, cited scenes

4 **Fix it and save**
A person stays in charge

5 **Produce**
Voice, cover, motion, film

Behrouz et al., Nested Learning, arXiv:2512.24695v1, CC BY 4.0. Figure 5, original page 30.

Done. Paper to explainer.

Prepared from the same project you just watched.

Learn at different speeds



Update-frequency categories, not measured hardware clocks

Authored schematic · not a measured physical simulation

How it fits together.

Each contract does one job. Pink = new.

Experiences

NEW

AgentBoundary
Hosts the agent in a Blazor page

AG-UI
Streams agent events and state to the UI

Agents + tools

AIAgent
Runs the multi-step authoring task

AIFunction
Lets the model call your C# method

Multimodal

ITextToSpeechClient
Reads each scene aloud

IRealtimeClient
Streams your voice into a brief

IImageGenerator
Makes the cover

Context

NEW

DocumentExtractionClient
Turns PDFs and images into pages, sections, tables

IngestionDocumentReader
Feeds them to token-sized chunking

Models + decisions

NEW

IChatClient
Talks to any model

RoutingChatClient
Picks the model for each request

Decisions API
Answers a typed question with probabilities

Shared Contracts: **Microsoft.Extensions.AI** · **Microsoft.Extensions.DataIngestion**

Core .NET: **Microsoft.ML.Tokenizers** · **System.Numerics.Tensors**

PROVIDERS

CLOUD

OpenAI (chat + Decisions)
Anthropic Claude (chat)
OpenAI Realtime (voice input)
Mistral OCR
GitHub Copilot SDK

LOCAL

PdfPig
Kokoro (voice-over)
Whisper.net (voice backup)
ONNX decision model

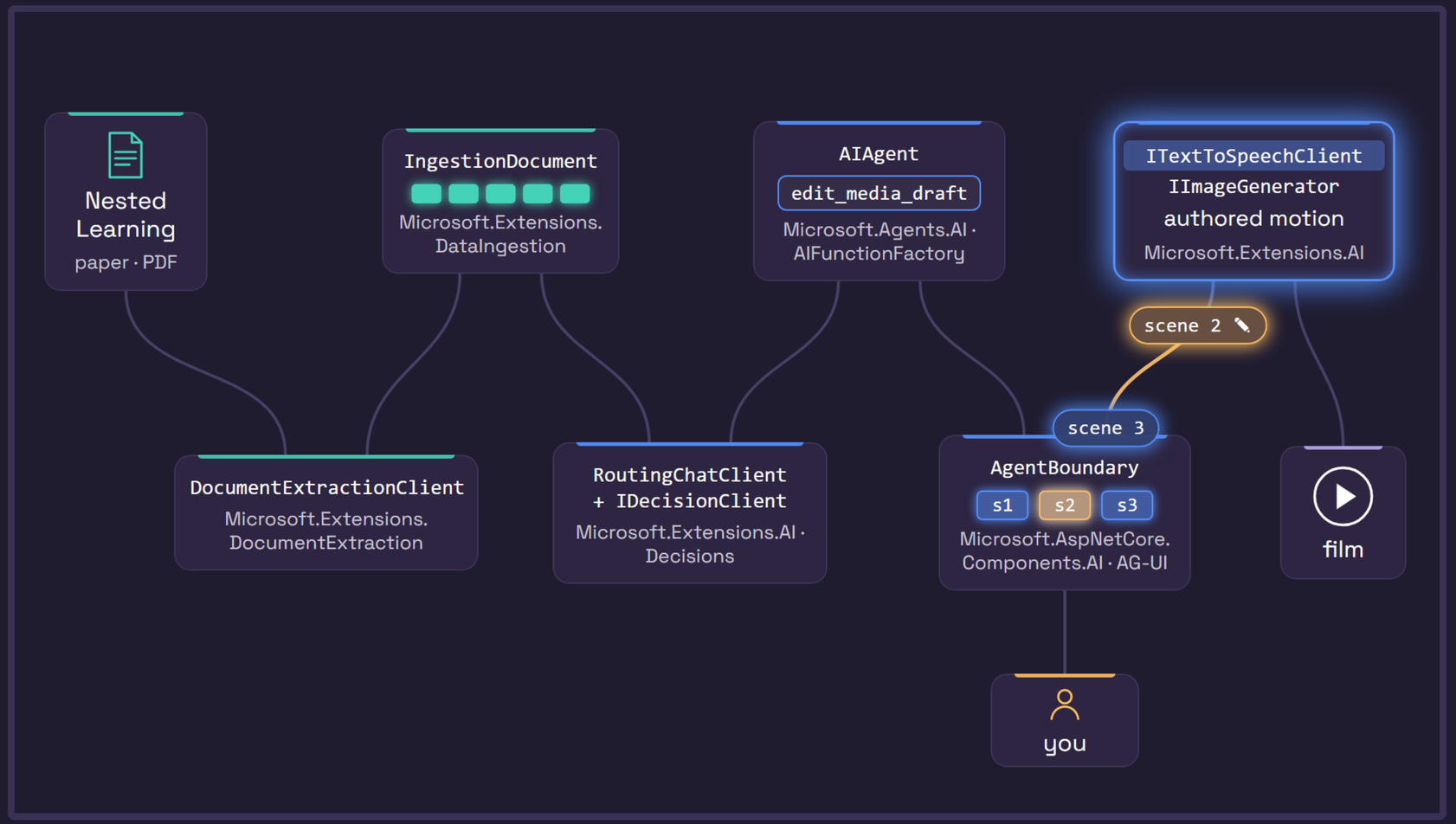
APP

WPF + Blazor Hybrid
Aspire AppHost
Policy, state, approval

Same contract.
Your choice of provider.

Follow one request.

PDF in, film out: every hop is a .NET contract.



Models Decisions Context Multimodal Tools Agents Experiences

Talk to any model.

GPT or Claude: each SDK hands back the same IChatClient.

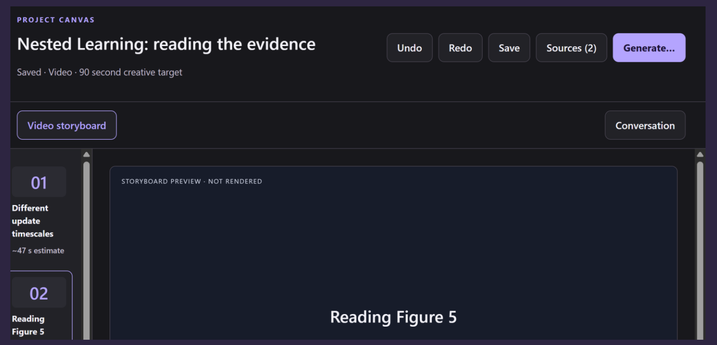
SOURCE / simplified from Planning.cs + FoundryAnthropic.cs

```
// Each official SDK has an AsIChatClient adapter
IChatClient gpt = new OpenAIClient(...)
    .GetResponsesClient().AsIChatClient("gpt-6.1-sol");

IChatClient claude = new AnthropicClient(...)
    .AsIChatClient("claude-sonnet-5-5");

// Same call, either model
var a = await gpt.GetResponseAsync(prompt);
var b = await claude.GetResponseAsync(prompt);
```

YOU SAW THIS



INPUT

A prompt, plus provider + model from config

OUTPUT

IChatClient: no OpenAI or Anthropic types downstream

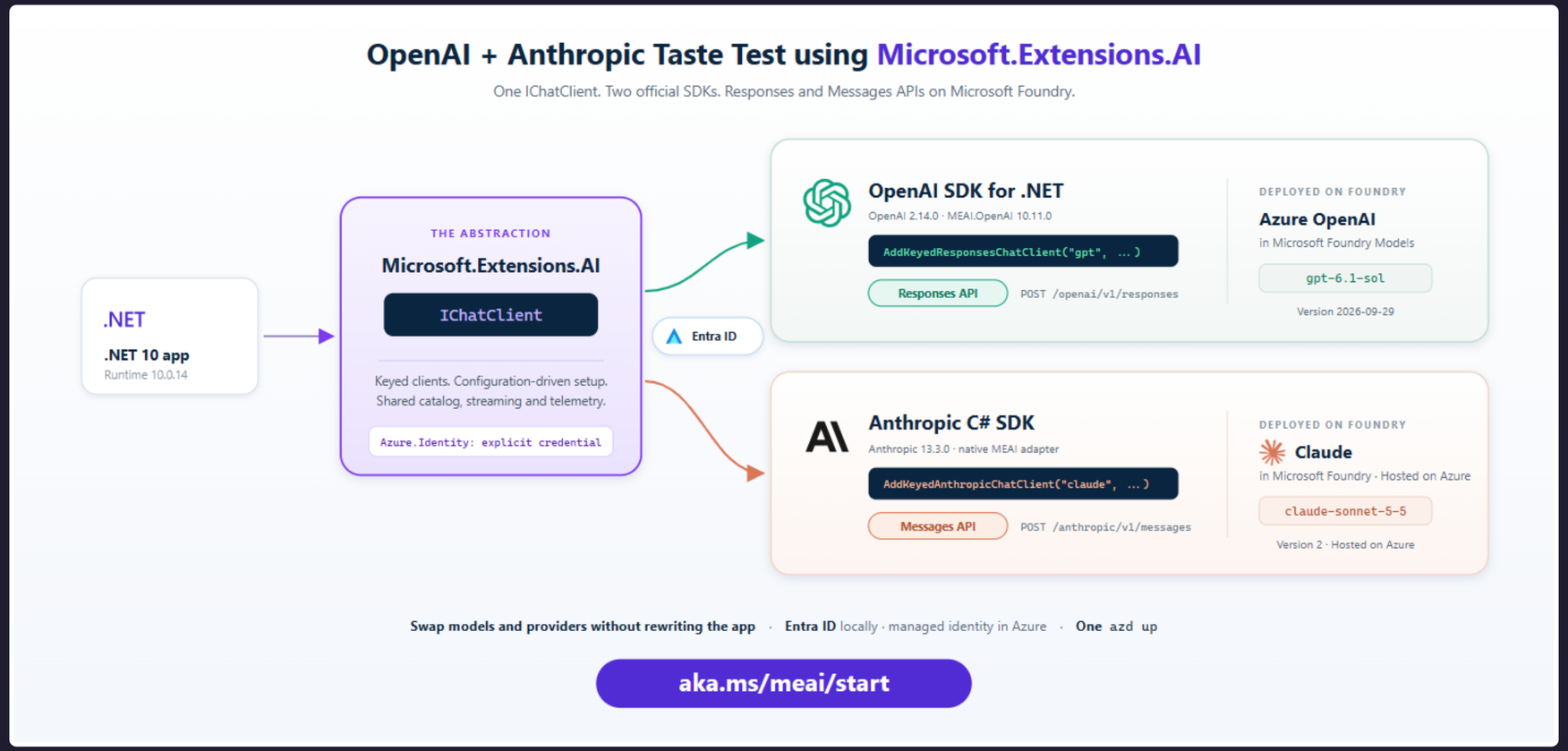
PACKAGES

Microsoft.Extensions.AI | Microsoft.Extensions.AI.OpenAI | Anthropic

Try it: GPT and Claude, side by side.

NEW SAMPLE

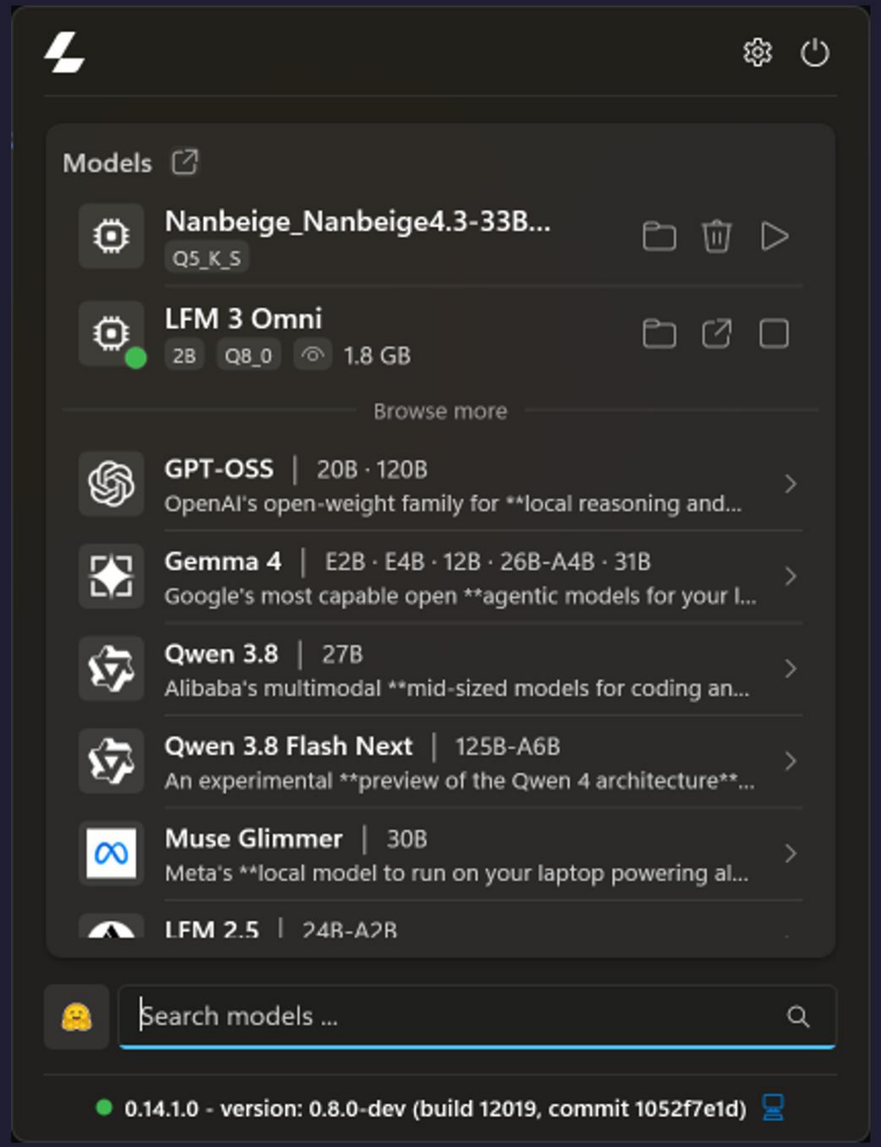
NEW sample: one IChatClient, two official SDKs.



Ask once. Both models answer. Vote blind, then reveal.

Llama on Windows

Now available in the Microsoft Store



A tray app to discover, download, and chat with local models.

Powered by .NET 10 + WinUI 3 + llama.cpp

- Integrate local models into your apps
- OpenAI-compatible endpoints
- **Open source. Download free at llama.app**
- Brought to you by Hugging Face + llama.cpp teams

Hugging Face + llama.cpp

Which model should answer?

NEW: RoutingChatClient. Your policy picks from a pool of IChatClients.

THE POLICY: A PLAIN RECORD / ChatRouting.cs

```
public sealed record ChatRoutePolicy(
    string Version, string DevelopmentEvidenceSha256,
    bool LocalOnly = true, bool RequireJson = false, bool AllowTools = false,
    int MaxInputCharacters = 24000, int MaxOutputTokens = 4096,
    double MinimumProbability = .7, double MinimumMargin = .2,
    string? UncertainRoute = null);
```

THE POOL: NAMED ROUTES

```
{ "name": "gpt",
  "model": "gpt-6.1-sol" },
{ "name": "claude",
  "provider": "anthropic",
  "model": "claude-sonnet-5-5" }
```

THE ROUTER: STILL JUST AN ICHATCLIENT

```
router = RoutingChatClient.Create(async (context, ct) =>
{
    var receipt = await Select(context.Messages,
                               context.ChatOptions, ct);
    var route = Routes.Single(r => r.Name == receipt.SelectedRoute);
    return new SelectedChatClient(clients[route.Name].Value, ...);
});
```

NEW · HOW IT DECIDES

1. The policy filters the pool:
   approved, local, JSON, tools, size.
2. Decisions scores
   what's left.
3. Not sure enough?
   Named fallback, or stop.

PACKAGES

Microsoft.Extensions.AI

Same router plans the video and draws the cover.

Ask a typed question. Get a typed answer.

NEW: OpenAI Decisions API. Same contract, cloud or local.

SOURCE / src/VideoProduction.Providers/ChatRouting.cs

```
var question = new ChoiceDecisionQuestion("route", "Choose the best eligible provider...",
    domain.Select(r => new DecisionCandidate(r.Name, r.Description)).ToArray());
var response = await decision.Value.GetResponseAsync(state, ..., [question], cancellationToken: ct);
var answer = response.Answers.Single() as ChoiceDecisionAnswer;   // choice + probabilities
```

CLOUD / OpenAiDecisionClient.cs

```
class OpenAiDecisionClient(DecisionsClient client, ...) : IDecisionClient
await client.CreateDecisionAsync(creation, ct);
```

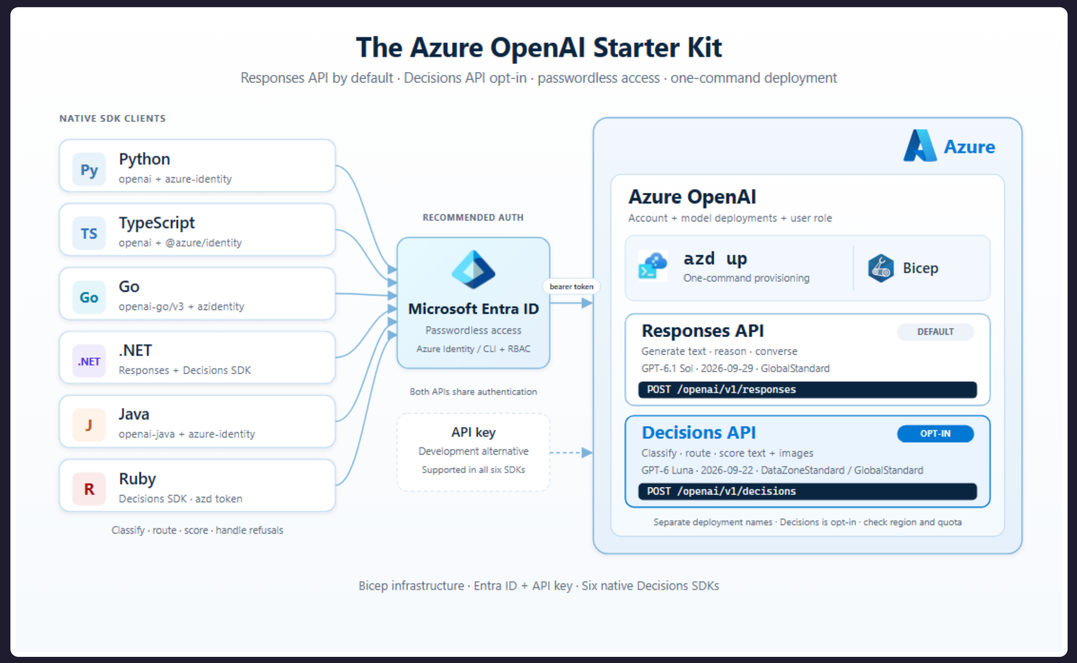
OpenAI

LOCAL / Julia/JuliaDecisionClient.cs

```
class JuliaDecisionClient : IDecisionClient   // local ONNX model
class JuliaTokenizer : Microsoft.ML.Tokenizers.Tokenizer
    _inner.Encode(input, ...)   // Tokenizers.HuggingFace inside
```

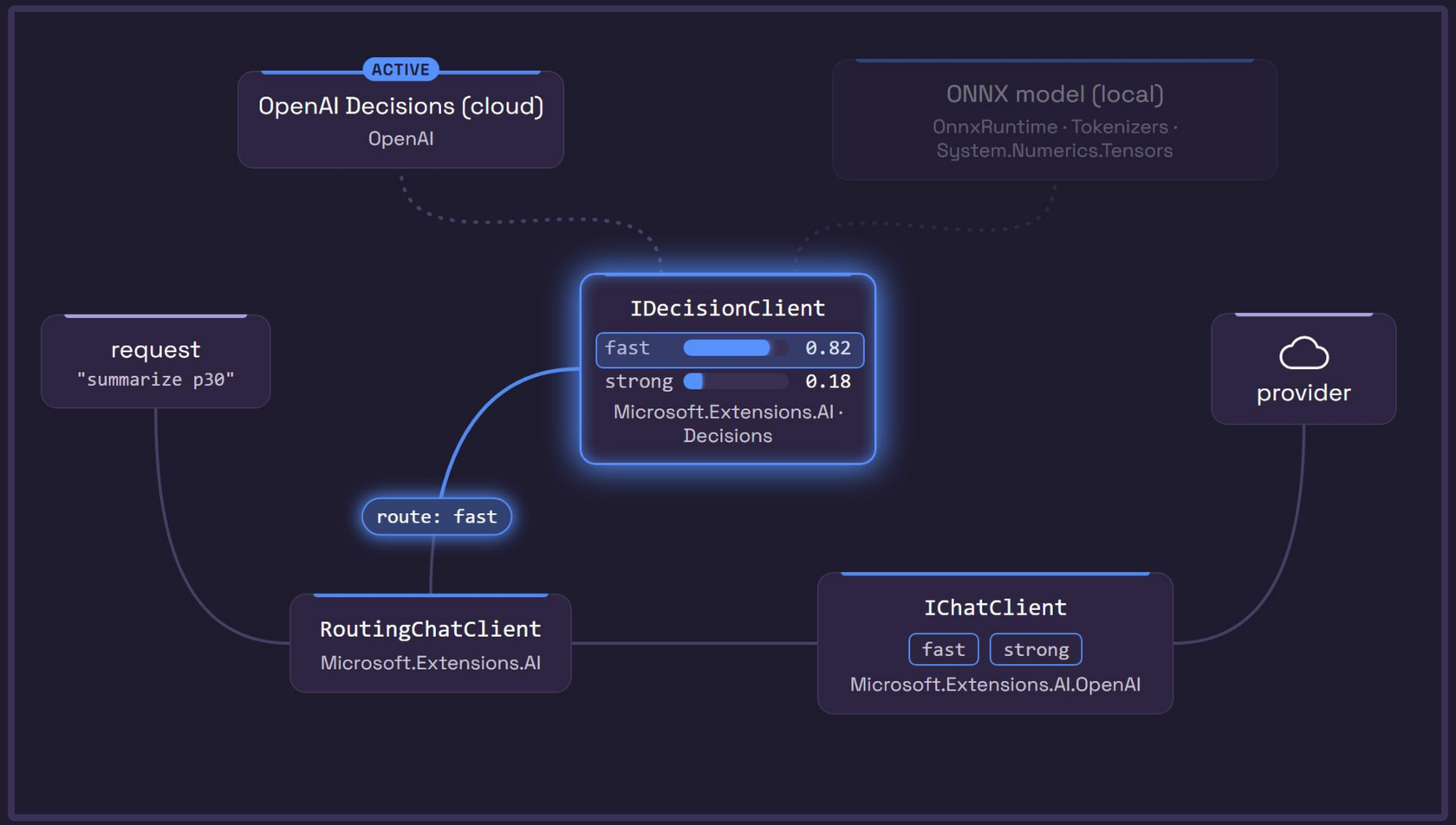
Microsoft.ML.OnnxRuntime | Microsoft.ML.Tokenizers | System.Numerics.Tensors

TRY IT **aka.ms/openai/start**



Decisions, in motion.

Policy filters. Decisions scores. RoutingChatClient routes.



Bring your context.

NEW: DocumentExtractionClient. One contract, three readers.

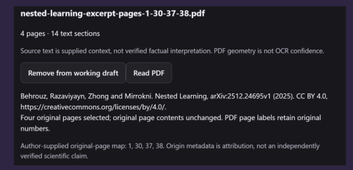
SOURCE / src/MediaStudio.Sources/DocumentSources.cs

```
this.pdf = pdf ?? new PdfPigDocumentExtractionClient();   // local

ocr = nativeOcr                                          // Mistral OCR (cloud)
    ?? new VisionLanguageModelDocumentExtractionClient(ocrChat!,
        new() { UseStructuredOutput = ocrCapabilities.StructuredOutput });

imageReader = new VisionLanguageModelDocumentExtractionClient(ocrChat,
    new() { Prompt = ImageReadingPrompt });   // any vision IChatClient
```

YOU SAW THIS



NEW · WHAT IT IS

One contract for reading documents: a file goes in, pages, sections, tables and images come out. Swap the reader, keep your code.

INPUT

4-page PDF excerpt (original pages 1, 30, 37, 38)

OUTPUT

4 pages, 14 sections, Figure 5, tables as Markdown

PACKAGES

Microsoft.Extensions.DocumentExtraction

CommunityToolkit.DocumentExtraction.{PdfPig, Mistral, VisionLanguageModel}

From pages to knowledge.

Extraction feeds Microsoft.Extensions.DataIngestion.

SOURCE / src/MediaStudio.Sources/DocumentSources.cs

```
class ExtractionIngestionReader(DocumentExtractionClient client, ...)
    : IngestionDocumentReader
{
    public override async Task<IngestionDocument> ReadAsync(Stream source,
        string identifier, string mediaType, CancellationToken ct = default)
    {
        LastExtraction = await client.ExtractAsync(
            source, identifier, mediaType, options, ct);
        return Bridge(LastExtraction);  // -> IngestionDocument
    }
}
```

YOU SAW THIS

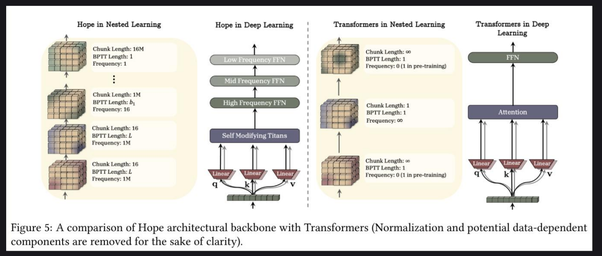


Figure 5: A comparison of Hope architectural backbone with Transformers (Normalization and potential data-dependent components are removed for the sake of clarity).

Source image payload

▸ Attribute an excerpt or source crop

INPUT

Extraction graph: pages, sections, figure bytes

OUTPUT

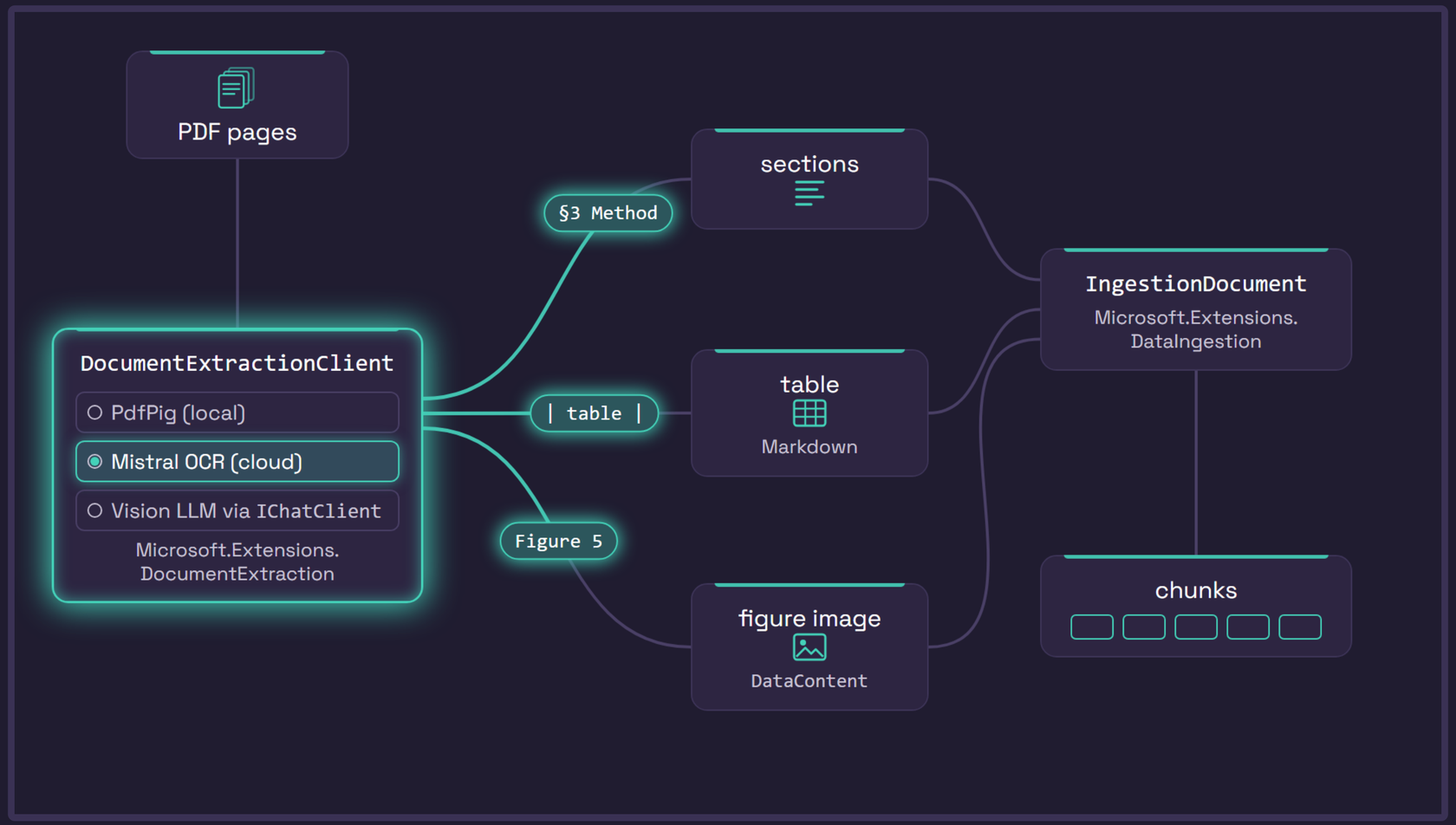
IngestionDocument for chunking; Figure 5 as DataContent to a vision model

PACKAGES

Microsoft.Extensions.DataIngestion · Microsoft.Extensions.DocumentExtraction · Microsoft.ML.Tokenizers

Context, in motion.

Swap the reader. Keep the contract.



Multimodal: hear it.

A voice for every scene, captions timed to it, and your voice as input.

SOURCE / src/Media.Narration/Speech.cs · StudioHybrid.Wpf/CloudSpeechInput.cs, RealtimeDictation.cs

```
response = await client.GetAudioAsync(displayText, new TextToSpeechOptions
{
    VoiceId = request.Voice, Language = request.Language,
    Speed = request.Speed, Volume = 1, AudioFormat = "wav"
}, ct);                                    // client = Kokoro, local


var session = await realtime.CreateSessionAsync(new RealtimeSessionOptions {
    SessionKind = RealtimeSessionKind.Transcription,
    TranscriptionOptions = new() { ModelId = "gpt-live-transcribe" } });
await session.SendAsync(new InputAudioBufferAppendRealtimeClientMessage(
    new DataContent(pcm, "audio/pcm")));    // words stream back as you talk
```

LISTEN



WHAT THEY DO

Text-to-speech: scene text in, voice-over out; captions are timed to that voice. Realtime: you talk, the brief types itself.

INPUT

Scene narration text · your live mic audio

OUTPUT

WAV narration (captions timed from it) · live brief

PACKAGES

Microsoft.Extensions.AI · Kokoro (local) · OpenAI Realtime

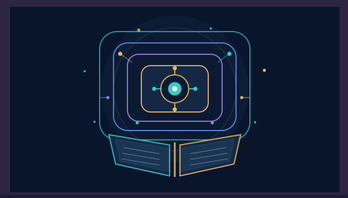
Multimodal: see it.

A chat model draws the cover, behind IImageGenerator.

SOURCE / src/VideoProduction.Providers/ChatSvgImageGenerator.cs

```
class ChatSvgImageGenerator(IChatClient chat, ...) : IImageGenerator
{
    // 1. ask a chat model for a picture, as SVG text
    response = await chat.GetResponseAsync(
        [new(ChatRole.System, "Return bare SVG only ..."),
         new(ChatRole.User, request.Prompt)], ...);
    // 2. check it, 3. render it to PNG with a pinned font
    var svg = SafeSvg.Validate(response.Text, size.Width, size.Height);
    return new ImageGenerationResponse([new DataContent(bytes, "image/png")]);
}
```

COVER



NO IMAGE MODEL HERE

IImageGenerator is the contract. Behind it, a chat model writes SVG; the app checks and renders it. A native image model can drop in later.

INPUT

Project title -> cover prompt

OUTPUT

One PNG cover, with the SVG kept as evidence

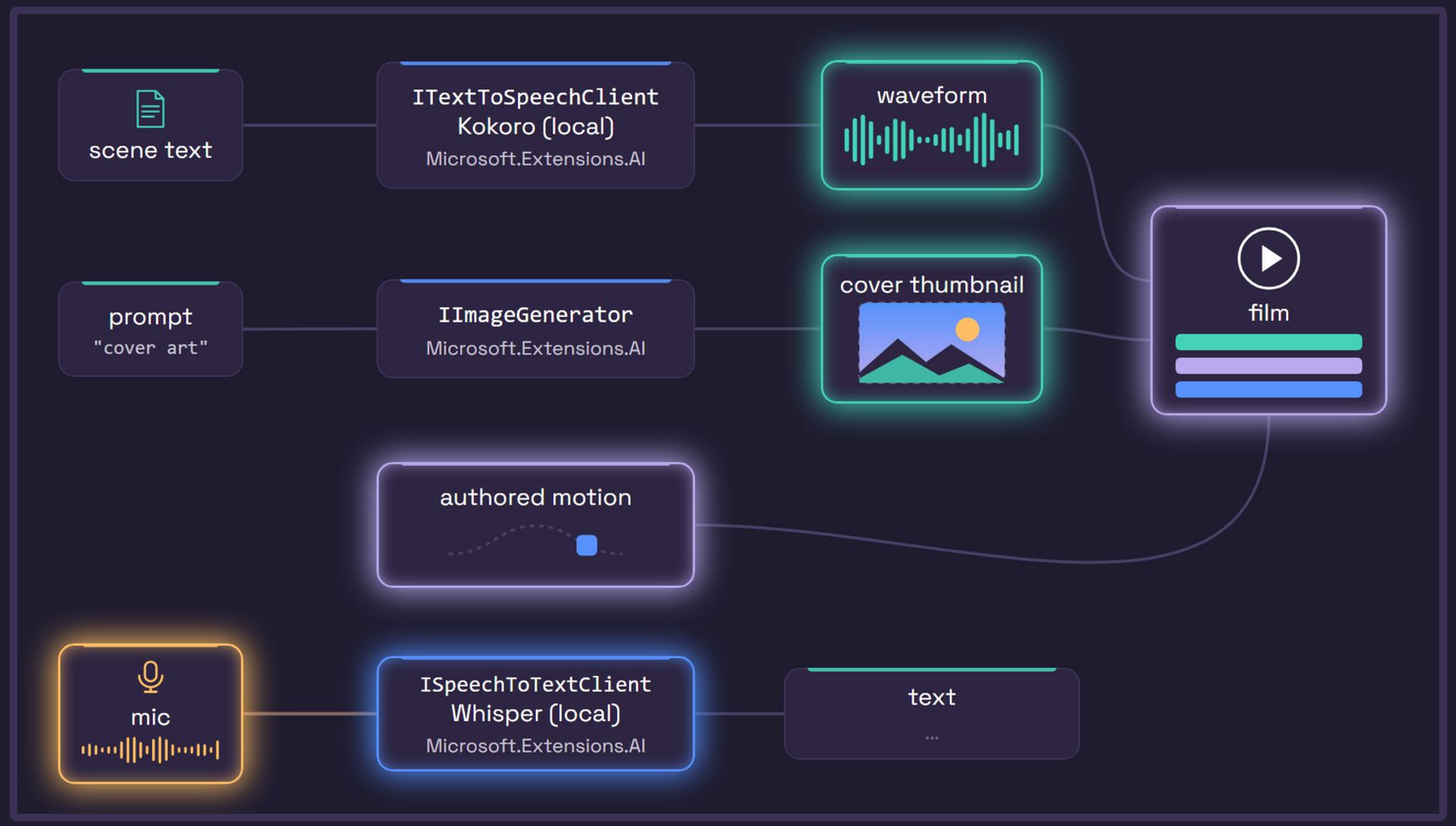
PACKAGES

Microsoft.Extensions.AI

Microsoft.Extensions.AI.OpenAI

Multimodal: how it becomes a film.

AI makes the parts. Plain app code puts them together.



AI CONTRACT

APP CODE, NOT AI

1 **Storyboard**
IChatClient: scene text

2 **Cover + scene images**
IImageGenerator

3 **Narration**
ITextToSpeechClient

4 **Captions**
timed from the narration

5 **Motion frames**
HTML + GSAP, rendered

6 **Film**
FFmpeg encodes the MP4

Give the model a tool.

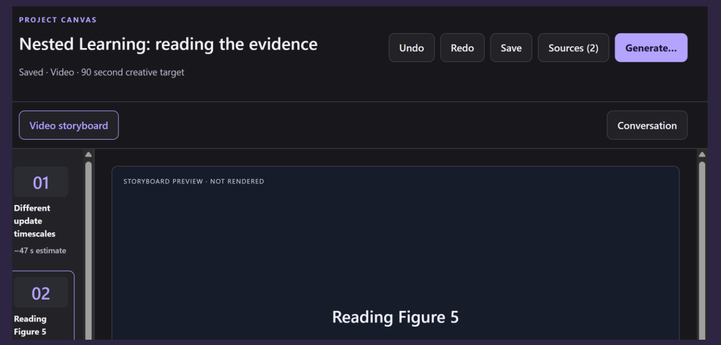
AIFunctionFactory turns a .NET method into a typed tool.

SOURCE / experiments/StudioHybrid.ApiProbe/MediaDraftTool.cs

```
static AIFunction Create(Delegate method, string name, string description) =>
    AIFunctionFactory.Create(method, new AIFunctionFactoryOptions {
        Name = name, Description = description,
        SerializerOptions = Json.Options });

tools.Add(Create(SetItemTarget, "set_item_target",
    "Set a creative duration target in seconds for one existing item."));
```

YOU SAW THIS



INPUT

A C# method and a description

OUTPUT

AIFunction with a JSON schema; returns a typed acknowledgement

PACKAGES

Microsoft.Extensions.AI

Make it an agent.

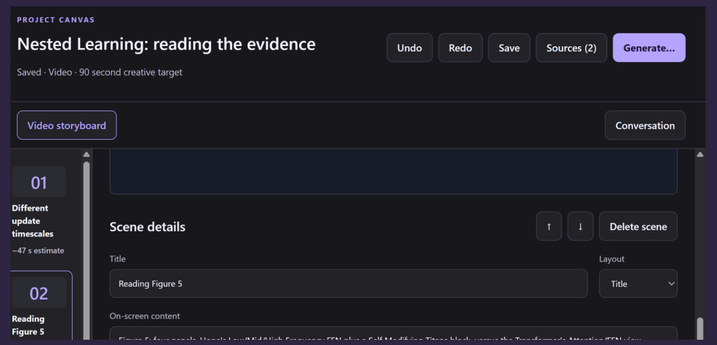
Microsoft Agent Framework: any IChatClient, or the GitHub Copilot SDK.

SOURCE / experiments/StudioHybrid.ApiProbe/CopilotAuthoringAgent.cs

```
public sealed class CopilotAuthoringAgent : AIAgent, IAsyncDisposable

if (chatClient is not null)                          // any IChatClient
    backend = chatClient.AsAIAgent(
        instructions: configuration.SystemMessage!.Content, name: Name);
else                                                 // GitHub Copilot SDK
    backend = copilot.AsAIAgent(configuration, ownsClient: false);
var session = await backend.CreateSessionAsync(cancellationToken);
```

YOU SAW THIS



INPUT

Brief + sources + the tools

OUTPUT

An agent session that streams tool calls and text

PACKAGES

Microsoft.Agents.AI | Microsoft.Agents.AI.GitHub.Copilot | GitHub.Copilot.SDK

New: put an agent in your UI.

NEW

Two new pieces. One protocol, one set of components.

THE PROTOCOL

AG-UI for .NET

Agent streams typed events
Run, text, tool call, state change
Works with any front end

AGUI.Server AGUI.Client

THE UI

Blazor AI components

<AgentBoundary> hosts an agent
Components render live state
The person edits; Save persists

Microsoft.AspNetCore.Components.AI

Agent ⇄ AG-UI events ⇄ Blazor UI: the agent proposes, the person decides.

Put the agent in the UI.

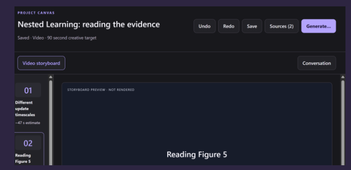
NEW: AG-UI for .NET + Blazor AI components.

SOURCE / VideoStudio.Client/Pages/Home.razor · StudioHybrid.Wpf/MediaWorkspace.cs

```
<AgentBoundary Agent="agent" @key="agent">
    <AuthorActivity @ref="activity" />
</AgentBoundary>

agentClient = new(new AGUIChatClientOptions
    { Transport = transport, JsonSerializerOptions = Json.Options });
var agent = new UIAgent<MediaDraftState>(agentClient, options => ...);
```

YOU SAW THIS



NEW · HOW IT CONNECTS

The client reads AG-UI events from the agent. UIAgent keeps the draft as typed state that both the agent and the person can change.

INPUT

AG-UI events: run, tool call, state delta

OUTPUT

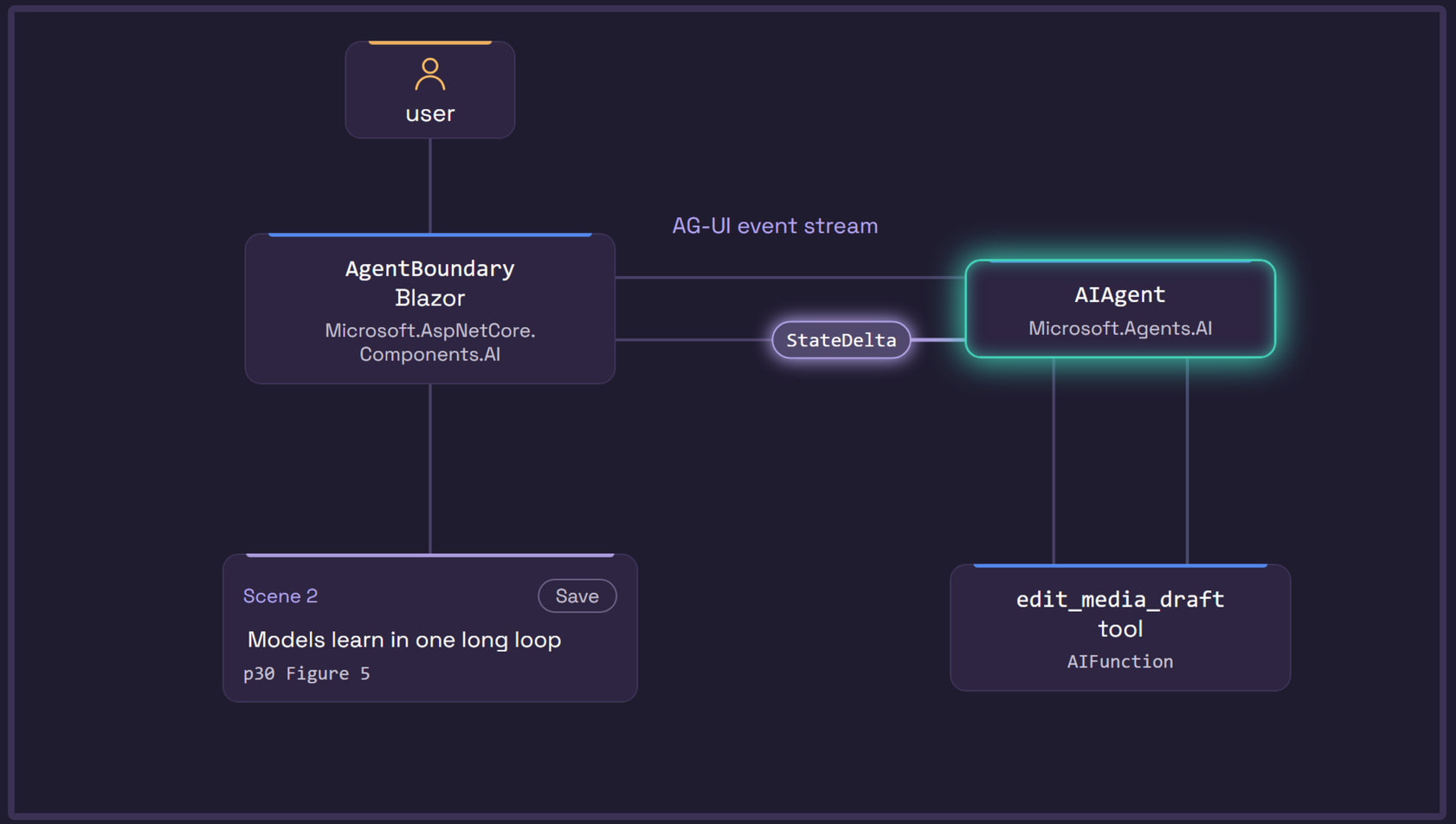
Live scene cards a person edits and saves

PACKAGES

- Microsoft.AspNetCore.Components.AI
- AGUI.Server
- AGUI.Client

The agent and the UI, in motion.

Propose, show, fix, save: the person stays in charge.



Shared foundations.

Contracts, ingestion, and core .NET primitives underneath.

MICROSOFT.EXTENSIONS.AI

The contracts

```
IChatClient
RoutingChatClient
ITextToSpeechClient
IRealtimeClient
IImageGenerator
AIFunction
DataContent
```

EXTRACTION + DATAINGESTION

Read + chunk the paper

```
DocumentExtractionClient reader = new
  VisionLanguageModelDocumentExtractionClient(chat);
var doc = await reader.ExtractAsync(pdf, ...);

tokenizer = TiktokenTokenizer
    .CreateForEncoding("cl100k_base");
new DocumentTokenChunker(
  new IngestionChunkerOptions(tokenizer)
  { MaxTokensPerChunk = 512 });
tokenizer.CountTokens(chunk.Content);
```

Microsoft.Extensions.DocumentExtraction

Microsoft.ML.Tokenizers

CORE .NET PRIMITIVES

Count, encode, compute

```
class JuliaTokenizer : Tokenizer

// softmax, local decision model
TensorPrimitives.Subtract<double>(values,
    TensorPrimitives.Max<double>(values), p);
TensorPrimitives.Exp<double>(p, p);
TensorPrimitives.Divide<double>(p,
    TensorPrimitives.Sum<double>(p), p);
```

System.Numerics.Tensors

Contracts over providers

Swap models, readers, voices.

Typed results over parsed text

Tools, decisions, extraction.

The app owns policy and state

Routes, bounds, receipts.

People approve

Edit, then explicitly save.

One app. The whole stack.

Small contracts. Real providers. You stay in charge.

NEW STARTER KIT

Talk to any model

IN THE DEMO

Every model call in the app

IChatClient · OpenAI + Anthropic SDKs

EXPERIMENTAL

Make the right decision

IN THE DEMO

Picked the model per request

RoutingChatClient · Decisions

PREVIEW

Bring your context

IN THE DEMO

Read the paper and Figure 5

DocumentExtraction · DataIngestion

EXPERIMENTAL

Go multimodal

IN THE DEMO

Voice, captions, cover, film

ITextToSpeechClient · IImageGenerator

Tools and agents

IN THE DEMO

Drafted 4 cited scenes

AIFunction · Agent Framework

NEW

Put it in the UI

IN THE DEMO

You fixed a scene and saved

AG-UI · Blazor AI components

Try it: **aka.ms/meai/start** GPT + Claude starter kit **aka.ms/openai/start** Azure OpenAI Starter Kit + Decisions

Go deeper with .NET + AI

From building intelligent apps to building apps with AI.

.NET + Foundry: Better Together

Bruno Capuano

Building the future of agentic UI with Blazor

Daniel Roth

What's new in the MCP C# SDK

Stephen Halter

Beyond the Agent Trace: Operating Foundry Agents with .NET Aspire

Shweta Iyer

Beyond Autocomplete: Getting the Most from GitHub Copilot for .NET

Wendy Breiding

Modernize .NET Applications with GitHub Copilot

Mika Dumont

dot.net/conf | aka.ms/meai/start | aka.ms/openai/start

Selected follow-on sessions. Titles from the live agenda; subject to change.

Keep going with .NET 11

Download it. Read what's new. Tell us what you think.

DOWNLOAD

Download .NET 11

get.dot.net

WHAT'S NEW

What's new in .NET 11

aka.ms/dotnet/11/whatsnew

SWAG

.NET Conf digital swag

aka.ms/dotnetconf/swag

SURVEY

2026 .NET Developer Survey



Scan to take it now

About 15 minutes. Helps shape the future of .NET.

aka.ms/dotnet/developer-survey

From this session: **aka.ms/meai/start** GPT + Claude starter kit **aka.ms/openai/start** Azure OpenAI + Decisions

Backup: the app decides what’s eligible.

Policy runs before any model is asked.

SOURCE / src/VideoProduction.Providers/ChatRouting.cs

```
var reason = !route.Approved ? "not-approved" : !route.Available ? "unavailable"
    : !validOrigin ? "invalid-origin"
    : !uri!.IsLoopback && (Policy.LocalOnly || !c.AllowRemote || uri.Scheme != "https")
        ? "remote-not-approved"
    : state.Json && !route.SupportsJson ? "json-not-supported"
    : state.Tools && !route.SupportsTools ? "tools-not-supported"
    : route.MaxInputCharacters < state.InputCharacters ? "input-capacity"
    : route.MaxOutputTokens < state.MaxOutputTokens ? "output-capacity" : "eligible";

Guard.Require(domain.Length > 0, "routing.no-eligible", ...);     // nothing is called
if (domain.Length == 1) return new(..., domain[0].Name, "sole-eligible", ...);
var uncertain = selected.Probability < Policy.MinimumProbability
    || selected.Probability - other < Policy.MinimumMargin;
var route = uncertain ? Policy.UncertainRoute : selected.Name;  // null: stop
```

Backup: what we’re careful to say.

- The paper’s results are the authors’ own; we explain, we don’t verify them.
- Tables came out of OCR as Markdown, not addressed cells.
- The vision model described Figure 5; a person corrected one reversed flow.
- The live paper-authoring turn used the Copilot agent directly, not the router.
- Motion visuals are authored schematics, not generative video; the cover is SVG drawn by a chat model.
- Decisions in MEAI are coming soon; Studio’s IDecisionClient previews the shape.
- Speech, image and Blazor AI types are experimental; Studio pins preview builds.